

Contesto di prodotto (cos'è RAWWAR)

- **RAWWAR** è un **gioco di gambling battle-royale** con **8 giocatori virtuali**. Ogni round dura **82–88s: 60s betting, 1s buffer, 14–20s playing** (eliminazioni video), **7s results**. Gira **24/7** in **stanze parallele** e indipendenti.
- **Scommesse supportate (RTP 95.50%)**: WINNER 7.64, FTD 7.64, TOP2 27, TOP3 53, EXACT2 55, EXACT3 335. Regola payout: $\text{stake} \times \text{odds}$, con **maxWin** per giurisdizione (default 3000.00).
- **Provably fair**: `serverSeedHash` pre-bet → reveal post-match; CSPRNG `HMAC_DRBG`; shuffle Fisher–Yates.
- **Roulette-style settlement**: prima mostriamo, poi regoliamo (show first, settle later). **Incomplete game**: VOID tutti o SETTLE completo.
- **Multi-room**: orchestrator assegna utenti alle stanze; ogni stanza ha il proprio engine e websocket.

Missione di sviluppo (cosa dobbiamo costruire ORA)

- Implementare **5 servizi**: **Frontend**, **Orchestrator**, **Backend del Gioco**, **RGS mock**, **Casino mock** — tutti nello stesso codebase, differenze solo via config (Pre-Prod $\rightleftharpoons$ Prod).
- Consegnare **contratti** (REST/WS) **vincolanti**, **catalogo errori completo** con mapping a 6 messaggi UX, **state machines**, **schema DB**, **sicurezza/compliance**, **test & CI/CD**.
- **Pre-Prod (24h)**: SQLite, Redis single, 10 stanze, 56 video, e2e smoke. **Prod 500k CCU**: Postgres HA, Redis Cluster, 560 video via CDN, mTLS, OTel, HPA.

Vincoli non negoziabili (da rispettare in revisione)

- **Niente rename o aggiunte arbitrarie** a: endpoint, eventi WS, payload, header richiesti, ****codici**** e loro mapping → **6 messaggi UX**.
- Pattern obbligatori: Repository/UnitOfWork, Outbox/Inbox, State Machine, Idempotency (48h), Circuit Breaker, DTO Validation, Correlation-ID, WS hardening.
- Compliance: GLI-19 (designed-to-comply), RG (reality check configurabile + ACK), AML 5y retention, GDPR, Geo/VPN fail-safe.

Cosa deve produrre il revisore (Claude/Gemini)

1. **Gap/ambiguità con PATCH inline** (mini-diff testuale) — niente discorsi generici.
2. **REGISTRY DIFF** per `REST_ERROR_REGISTRY` e `WS_ERROR_REGISTRY` (aggiunte/rimozioni di codici ammessi per endpoint/evento).

3. **Test mancanti:** contract, load, fault-injection, DR.
4. **CHG-REQUEST** (opzionale) solo se un caso non è coperto da RW-SYS-000 (fallback), senza modificare la Bibbia.

Formato di risposta obbligatorio

- **A) BLOCCANTI (S0)** — RIF/PROBLEMA/PATCH/RAZIONALE/IMPATTO.
- **B) IMPORTANTI (S1)** — RIF/PROBLEMA/PATCH/RAZIONALE.
- **C) NICE-TO-HAVE (S2)** — lista sintetica.
- **D) REGISTRY DIFF (VINCOLANTE)** — delta per endpoint/evento.
- **E) TEST CHE MANCANO** — puntuale.
- **F) CHG-REQUEST (opz.)** — proposta singola formale.

Regola d'oro: non cambiare i contratti, non introdurre nuovi codici RW-*; se servono, proponi una CHG-REQUEST.

RAWWAR — Architectural Bible (v3, Extended “zero scuse”)

Obiettivo: documento prescrittivo, completo e deterministico per implementare RAWWAR senza ambiguità. **Vincoli:** unico codebase, differenze Pre-Prod/Prod SOLO via config; nessuna libertà sui nomi di endpoint, eventi WS, codici errore, strutture dati.

§0 — Data Schema Canonico (fonte unica)

Questa sezione **congela** nomi, fasi, eventi, endpoint, envelope e costanti. Tutto il documento **deve** riferirsi a questi valori.

§0.1 (append nota)

+ Nota: negli eventi ``match:*``, il campo ``playerId`` rappresenta l’**indice del concorrente (1..8)** della stanza, non l’ID account del bettor.

+ Il naming resta immutato per compatibilità; vedere anche Appendice B
``match_elimination.playerId: integer [1..8]``.

§0.1 Identificatori (case-sensitive)

- `playerId` — utente autenticato (REST, DB, RGS).

- `playerIndex` — indice concorrente (1..8) usato negli eventi WS ``match:*``.
- Nota: in payload WS il campo si chiama ancora ``playerId`` per compatibilità, ma semantica = `playerIndex`.
- `roomId` — stanza logica (namespace WS).
- `matchId` — round/partita in una stanza.
- `roundId` — identificatore round verso RGS.
- `betId` — scommessa singola.
- `idempotencyKey` — chiave idempotenza (POST/Batch).
- `correlationId` — tracciamento end-to-end.

§0.2 Phases (ordine e durata)

Engine: WAITING → BETTING(60s) → LOCK(~1s) → PLAYING(14–20s) → RESULTS(7s) → SETTLEMENT.

Esposizione WS al FE: BETTING, PLAYING, RESULTS.

§0.3 Tipi di scommessa & odds (RTP fisso)

RTP **95.50%**. Moltiplicatori ufficiali:

- WINNER → **7.64**
- FTD (First to Die) → **7.64**
- TOP2 → **27**
- TOP3 → **53**
- EXACT2 → **55**
- EXACT3 → **335**

§0.4 Eventi WebSocket (nomi esatti)

Client → Server: `room:join`, `bet:place`, `bet:cancel`, `heartbeat`, `sync:state`

****Server → Client****: ``room:joined``, ``room:playerCount``, ``match:phase`` (include sempre ``msRemaining``), ``match:sequence_hash``, ``match:elimination``, ``match:results``, ``bet:accepted``, ``bet:rejected``, ``bet:canceled``, ``bet:settled``, ``balance:updated``, ``error``, ``sync:ack``

§0.5 Endpoint REST (metodo + path)

Orchestrator: POST /assign, GET /rooms, POST /rooms, DELETE /rooms/{roomId}

Backend del Gioco (v1): GET /api/v1/match/current, POST /api/v1/bets/place, DELETE /api/v1/bets/{betId}, GET /api/v1/history/recent, GET /api/v1/paytable, GET /api/v1/provably-fair/{matchId}

RGS: POST /session/validate, POST /round/start, POST /transaction/debit, POST /transaction/credit, POST /round/end

Casino (mock) & Webhook: POST /wallet/debit, POST /wallet/credit, POST /limits/check, POST /rgs/playerExcluded, POST /rgs/balanceChanged

Admin/Backoffice: POST /admin/v1/rooms/{roomId}/drain, POST /admin/v1/matches/{matchId}/void, GET /admin/v1/health

§0.6 Envelope errore & formati

- **REST:** { "error": { "code": "RW-AAA-000", "message": "...", "correlationId": "..." } }
- **WS (error):** { code, message, correlationId }
- **Money:** stringa decimale a 2 cifre (es. "10.00"), arrotondamento **banker's** lato server **prima** della persistenza.
- **Timestamp:** ISO8601 UTC (es. "2025-08-17T12:00:00Z").

§0.7 Costanti / proprietà globali

RTP = "95.50" (fisso, help UI).

WS_SUBPROTOCOL = rawwar.v1.

MAX_WIN_DEFAULT = "3000.00" (override per giurisdizione).

IDEMPOTENCY_WINDOW = 48h (REST critici e batch RGS).

BET_RATE_LIMIT_PER_SESSION = 1 richiesta / 2s.

RGS_BATCH_MAX = 200 (Pre-Prod: 50; Prod: 200).

ROUND_CLOSE_JITTER_MS = ±2000.

CURRENCY = EUR.

WS_HEARTBEAT = ping/pong 30s, disconnect 60s.

REALITY_CHECK_MINUTES = configurabile per utente (min 30, max 120; default 60).

`SINGLE_ACTIVE_SESSION = true` (1 sessione per utente alla volta).

`WS_MAX_PAYLOAD_BYTES = 65536` (64KB).

0) Principi inviolabili (non negoziabili)

- **Codebase unico (monorepo)**; nessun `if (prod)`. Differenze SOLO via env/helm/feature flags.
- **Naming fissato (case-sensitive)**: `playerId`, `playerIndex`, `roomId`, `matchId`, `roundId`, `betId`, `idempotencyKey`, `correlationId`.
- **Formati**: timestamp ISO8601 UTC; **money come stringa** decimale (es. "10.00"); indici/posizioni come interi.
- **Provably Fair**: HMAC_DRBG (NIST SP 800-90A), `serverSeedHash` pre-bet, **reveal** post-match, shuffle Fisher–Yates.
- **Event-Driven + Outbox/Inbox** per ogni side-effect finanziario (**exactly-once effect** documentato).
- **Idempotency obbligatoria** via header `Idempotency-Key` (finestra 48h); rifiuto se assente su POST critiche.
- **Show first, settle later** (visual FE → poi credit RGS).
- **Incomplete game**: solo **VOID tutti** o **SETTLE completo** (mai parziali) secondo regole dichiarate.
- **Batching RGS $\leq 200:1$ + Jitter close $\pm 2s$** tra stanze.
- **SLO Prod**: availability 99.95%; p95 debit/credit < 200ms; p95 bet placement < 200ms; **clock drift $\leq 1s$** .

1) Monorepo, standard, versioning e config

1.1 Struttura monorepo

/apps	
/frontend	(React 18 + TS + Zustand + Vite)
/orchestrator	(Node 20 + TS + Fastify)
/backend-gioco	(NestJS 10 + Fastify + TS)
/rgs	(Node 20 + TS mock realistico)
/casino	(Node 20 + TS mock realistico)
/packages	
/shared-types	(types TS + zod/typebox schemas + JSON Schema)

/shared-lib	(crypto, id, logger, http, ws,
errors, i18n)	
/config	(loader config, schema env, feature
flags)	
/eslint-config	(preset lint)
/spec	
/openapi	(yaml REST)
/asynapi	(yaml WS room)
/sql	(DDL Postgres/SQLite)
/k8s	(helm charts)
/docs	(diagrammi, runbook, DR plan,
compliance)	
/.env.example	
/Makefile	
/README.md	

1.2 Standard globali

- **TS strict: true;** vietato any; ESLint (airbnb-base + custom), Prettier enforced in CI.
- **Conventional Commits;** trunk-based; semver; API versioning **/api/v1/** (REST) e subprotocol WS **rawwar.v1**.
- **UTC only; log JSON** (pino) con **correlationId** propagato HTTP+WS.
- **Rounding:** arrotondamento commerciale “Banker’s rounding” a 2 decimali server-side prima di persistenza.
- **Currency:** EUR (estendibile), formato stringa con 2 decimali.

1.3 Config & ambienti (solo via config)

.env.example (estratto; in Prod segreti in KMS/HSM):

```

NODE_ENV=development
LOG_LEVEL=info
REDIS_URL=redis://localhost:6379
DB_URL=sqlite:./dev.db           # Prod: postgres://
user:pass@host:5432/rawwar
CDN_BASE_URL=https://cdn.example.com/rawwar
PUBLIC_WS_BASE=wss://rooms.example.com
JWT_PUBLIC_KEY_BASE64=...
JWT_PRIVATE_KEY_BASE64=...
HMAC_SECRET_BASE64=...
RGS_BASE_URL=http://rgs:3000
CASINO_BASE_URL=http://casino:3001

```

RGS_BATCH_MAX=200
200
ROUND_CLOSE_JITTER_MS=2000
BET_RATE_LIMIT_PER_SESSION="1/2s"
REALITY_CHECK_MINUTES=60
utente, min 30, max 120
SINGLE_ACTIVE_SESSION=true
WS_MAX_PAYLOAD_BYTES=65536
WS_PING_INTERVAL_MS=30000
RPO_MINUTES=5
RTO_MINUTES=15
FEATURE_FLAG_SYNCSTATE=true
FEATURE_FLAG_VIDEO_FALLBACK_TEXT=true

Pre-Prod: 50 | Prod:

±2s

configurabile per

Pre-Prod (24h) vs Prod (500k CCU)

Voce Pre-Prod Prod		
DB	SQLite file	Postgres HA (Patroni), pgBouncer, partition matchId
Redis	Single	Redis Cluster (≥6 shards), AOF
Stanze	10 × 100	200 × 2,500
Video	56 clip locali/CDN dev	560 clip su CDN globale (signed URLs)
TLS/Secrets	self-signed/env	TLS 1.3 ovunque, mTLS intra-svc, KMS/HSM
CI/CD	build + e2e smoke	full pipeline (SBOM, SCA, DAST, sign, blue/green)
Observability	logs + metrics base	OTel full, Grafana/Loki/Tempo, alert SLO
Timeouts	HTTP 2s	HTTP 5s; RGS 3s

1.4 Deprecation policy

- /v1 congelato per 12 mesi; nuove versioni su /v2 con header Deprecation/Sunset. Nessun breaking change in minor.

2) Pattern non-negoziabili

- **Repository + UnitOfWork**: accesso DB transazionale; rollback ACID; una Unit per richiesta.
- **Outbox/Inbox**: persist write + publish in transazione; consumer idempotente; DLQ con retry policy esponenziale.
- **State Machine (Backend del Gioco)**: WAITING → BETTING → LOCK → PLAYING → RESULTS → SETTLEMENT.

- **Circuit Breaker + Exponential Backoff (jittered)**: RGS/Casino; fallback; breaker half-open policy.
- **DTO Validation** (zod/typebox) su **tutti** ingressi (REST/WS). 400/error immediato.
- **Correlation-ID**: header `X-Correlation-Id` e campo `ws`; obbligatorio in log, metrics, traces.
- **Rate limiting**: per sessione (1 bet/2s), per IP (WAF/gateway), per endpoint.
- **Clock sync**: NTP/Chrony su nodi; server rifiuta skew > 60s con `CLOCK_SKEW`.

3) Domain model & regole di convalida

3.1 Entità

- **Player** { playerId, jurisdiction, limits }
- **Room** { roomId, maxPlayers, status }
- **Match** { matchId, roomId, phase, serverSeedHash, serverSeed?, sequence[], createdAt, closedAt? }
- **Bet** { betId, matchId, playerId, type, selection[], stake, status }
- **Settlement** { id, matchId, playerId, amount, status }

3.2 Bet Types & Odds (RTP 95.50% certificato)

- WINNER 7.64:1 → selection=[p], p∈[1..8]
- FTD (First to Die) 7.64:1 → selection=[p], p∈[1..8]
- TOP2 27:1 → selection=[a,b] con a≠b, a,b∈[1..8]
- TOP3 53:1 → selection=[a,b,c] distinti
- EXACT2 55:1 → selection=[first,second] distinti
- EXACT3 335:1 → selection=[first,second,third] distinti

Convalida:

- valori distinti per array;
- range 1..8;
- EXACT* rispettano ordine;

- TOP* ordine irrilevante (normalizzare in sort crescente prima di persistenza).

3.3 Rounding & payout

- $\text{payout} = \text{stake} \times \text{odds}$, arrotondato a 2 decimali (banker's) prima di credito.
- maxWin per giurisdizione (default 3000.00) → errore MAX_WIN_EXCEEDED.

4) Contratti di comunicazione (REST + WS)

4.1 Frontend ↔ Orchestrator (REST)

- POST /assign
Req { "playerId": "u_123", "jurisdiction": "MGA" }
Res 200 { "roomId": "r_42", "wsEndpoint": "wss://rooms/r_42" }
Errors: RW-ORCH-001/002/003/006, RW-GEO-002
Headers: Authorization, X-Correlation-Id
- GET /rooms → telemetria (opzionale FE)
Res [{ "roomId": "r_42", "players": 2310, "status": "ACTIVE" }]

4.2 Frontend ↔ Backend del Gioco (REST)

- GET /api/v1/match/current
Res
{ "roomId": "r_42", "matchId": "m_...", "phase": "BETTING", "msRemaining": 42000, "serverSeedHash": "hex" }
- POST /api/v1/bets/place (*idempotente, header Idempotency-Key richiesto*)
Req { "matchId": "m_...", "type": "WINNER", "selection": [3], "stake": "10.00" }
Res 201 { "betId": "b_...", "status": "PENDING" }
Errors: RW-GAME-001/002/003/004/005/006/007/008/009/010/011/017/018, RW-RGS-001/002/003, RW-NET-009
- DELETE /api/v1/bets/{betId} (*solo in BETTING*)
Res 204
Errors: RW-GAME-019/001/018, RW-AUTH-002
- GET /api/v1/history/recent
Res
[{ "betId": "b_...", "matchId": "m_...", "type": "WINNER", "se

```
lection":  
[3], "stake": "10.00", "result": "WIN", "payout": "76.40" } ]
```

- GET /api/v1/paytable
Res { "rtp": "95.50", "odds":
{ "WINNER": "7.64", "FTD": "7.64", "TOP2": "27", "TOP3": "53", "EX
ACT2": "55", "EXACT3": "335" }, "minBet": "0.20", "maxBet": "200.
00", "maxWin": "3000.00" }
- GET /api/v1/provably-fair/{matchId}
Res
{ "serverSeedHash": "hex", "serverSeed": "base64", "clientSee
dHash": "hex", "sequence": [...], "verifyUrl": "..."} }

Envelope errore REST (standard) -> Envelope errore REST: vedi §0.6 (formato canonico).

4.3 Frontend ↔ Backend del Gioco (WebSocket room-scoped)

Endpoint: wss://{host}/ws/{roomId}

Subprotocol: rawwar.v1

Hardening: origin check; whitelist subprotocol; **max payload 64KB**; **compression disabilitata**.

Heartbeat: ping/pong 30s; disconnect 60s.

Reconnect backoff: 0.5s → 1s → 2s → 5s → 10s (cap 30s).

Client → Server

- room:join { roomId, token, correlationId } → Ack
room:joined
- bet:place
{ matchId, type, selection, stake, idempotencyKey, correlation
Id } → bet:accepted/bet:rejected
- bet:cancel { betId, correlationId } → bet:canceled/error
- heartbeat { ts }
- sync:state { lastEventId: "evt-123" } → server invia delta eventi mancanti
(id monotoni evt-...)

Server → Client

- room:joined { roomId }
- room:playerCount { count }
- match:phase { phase: "BETTING" | "PLAYING" | "RESULTS",
msRemaining }

- `match:sequence_hash { serverSeedHash }`
- `match:elimination { index, playerId }`
- `match:results { positions:[...8], serverSeed:"base64" }`
- `bet:accepted { betId, status:"PENDING" }`
- `bet:rejected { betId, reason:"RW-..." }`
- `bet:canceled { betId, status:"CANCELED" }`
- `bet:settled { betId, result:"WIN"|"LOSE", payout }`
- `balance:updated { balance }`
- `error { code:"RW-...", message, correlationId }`
- `sync:ack { fromEventId:"evt-124", toEventId:"evt-141" }`

Regole WS: ogni richiesta deve avere `correlationId`; le risposte lo riflettono. Errori solo via `error` o campo `reason`.

4.4 Backend del Gioco ↔ RGS (REST, batch)

Base URL: `${RGS_BASE_URL}`

Headers: `Authorization`, `X-Correlation-Id`, `Idempotency-Key`

- `POST /session/validate → { playerId, casinoId, jurisdiction } → { status:"VALID", limits:{...}, currency:"EUR" }`
- `POST /round/start { roundId, gameId:"RAWWAR", ts } → { status:"OK" }`
- `POST /transaction/debit Req { "items": [{ "playerId","amount","currency","roundId","idempotencyKey" }] } Res { "results": [{ "status":"APPROVED"|"REJECTED", "reason?":"RW-RGS-*","txId?","newBalance?" }] }`
- `POST /transaction/credit (batch simmetrico)`
- `POST /round/end { roundId, result:"OK" } → { status:"OK" }`

Idempotency: richiesto (header). Finestra 48h.

4.5 RGS ↔ Casino (Mock)

- `POST /wallet/debit{ playerId, amount, currency, idempotencyKey } → { status:"APPROVED"|"REJECTED", reason?, newBalance? }`
- `POST /wallet/credit{ ... }`
- `POST /limits/check{ playerId, stake } → { ok:true|false, reason? }`
- Webhook `POST /rgs/playerExcluded{ playerId, reason, ts }`
- Webhook `POST /rgs/balanceChanged{ playerId, newBalance, ts }`

5) Catalogo errori (interni RW-*) e Facade UX

5.1 Facade UX (mostrare SOLO questi 6 messaggi)

1. **UX_CONN_ISSUE** — “Connessione instabile. Riprova.”
2. **UX_SESSION_INVALID** — “Sessione non valida. Accedi di nuovo.”
3. **UX_LIMITS** — “Hai raggiunto i limiti di gioco.”
4. **UX_INSUFFICIENT_FUNDS** — “Saldo insufficiente.”
5. **UX_SERVICE_BUSY** — “Servizio temporaneamente occupato.”
6. **UX_CONTENT_UNAVAILABLE** — “Contenuto non disponibile.”

5.2 Tassonomia completa RW-codes

- **RW-SYS-000 (INTERNAL_ERROR: fallback ultimo miglio → mappa a UX_SERVICE_BUSY; usato solo per eccezioni non classificate; target in Prod ≈ 0%)**
- **FE:** RW-FE-001...008
- **VID:** RW-VID-001...008
- **NET/WS:** RW-NET-001...010
- **AUTH:** RW-AUTH-001...006
- **ORCH:** RW-ORCH-001...006
- **GAME:** RW-GAME-001...020
- **RGS:** RW-RGS-001...015

- **CAS:** RW-CAS-001...010
- **DB/CACHE/QUEUE:** RW-DB-001...005, RW-CACHE-001...002, RW-Q-001...004
- **CFG/SEC/GEO/RG:** RW-CFG-001...002, RW-SEC-001...002, RW-GEO-001...002, RW-RG-001...002

5.3 Mapping RW-code → Facade UX (vincolante)

- **UX_CONN_ISSUE** ← RW-NET-*, RW-VID-003/007, RW-Q-002, RW-DB-003
- **UX_SESSION_INVALID** ← RW-AUTH-001/002/003, RW-RGS-003, RW-GEO-002, RW-GAME-020
- **UX_LIMITS** ← RW-RGS-002, RW-CAS-003, RW-RG-002, RW-GAME-008
- **UX_INSUFFICIENT_FUNDS** ← RW-RGS-001, RW-CAS-002
- **UX_SERVICE_BUSY** ← RW-NET-009, RW-RGS-008, RW-DB-001/003, RW-Q-003, RW-GAME-015
- **UX_CONTENT_UNAVAILABLE** ← RW-VID-001/002/005/006/008, RW-ORCH-002/006

(messaggi utenti localizzati in ``)

6) Frontend (React + TS + Zustand)

6.1 Struttura

```
/apps/frontend/src/
  app/
  components/
  hooks/
  pages/
  routing/
  services/
    api/ (OpenAPI client)
    ws/
  state/
    user.store.ts
    room.store.ts
    match.store.ts
    bets.store.ts
    ws.store.ts
```

```
video.store.ts
utils/
assets/
styles/
```

6.2 Slice di stato (con error/UX)

- `user.store: { playerId, token, balance, jurisdiction, limits, sessionStatus, lastUXError? }`
- `room.store: { roomId, players, status }`
- `match.store: { matchId, phase, msRemaining, serverSeedHash, sequence?, results?, lastEventId?, lastError? }`
- `bets.store: { cart[], pending[], confirmed[], settled[], lastError? }`
- `ws.store: { connected, status, lastPing, reconnectAttempts, lastError? }`
- `video.store: { status, bitrate, lastError? }`

6.3 UX error handling

- Mostrare SOLO 6 messaggi facade UX; codici RW-* in console log + Sentry (o equivalente) con `correlationId`.

6.4 Performance & a11y

- **Budget:** TTI < 3s (4G buono), memoria < 150MB, p95 input→send < 50ms.
- A11y: contrasto WCAG AA, focus visibile, scorciatoie tastiera (scommessa rapida), riduzione animazioni.

6.5 Help screen (conformità)

- Paytable + **RTP 95.50%** (link certificato), min/max bet, max Win per mercato, tutorial provably fair.

6.A) Frontend — Addendum VINCOLANTE (sostituisce e dettaglia §§6.1–6.9)

Nota: questa sezione è **prescrittiva** e **sostituisce** i sottoparagrafi di §6. Tutte le funzioni, file e props qui sotto sono **obbligatorie**.

A1) Contract di componenti (props e responsabilità)

- **RoomHeader** { roomId: string; players: number; wsStatus: 'OK' | 'RECONNECTING' | 'BACKOFF' | 'FAILED' }
 - Mostra lo stato WS con icona semaforo. Nessun side-effect.
- **Timer** { msRemaining: number; phase: 'BETTING' | 'PLAYING' | 'RESULTS' }
 - Conteggio ogni 200 ms; blocco a zero; non mostra barra durante PLAYING/RESULTS.
- **OddsGrid** { odds: Record<BetType, string>; onSelect: (type: BetType, selection: number[]) => void }
 - Rende disabilitato se selectCanBet===false.
- **BetSlip** { cart: BetItem[]; onPlace: (item: BetItem, idempotencyKey: string) => void; onRemove: (betId: string) => void }
 - Disabilita il bottone Place 1 secondo dopo click o fino a bet:accepted/rejected.
- **VideoBoard** { status: 'BUFFERING' | 'PLAYING' | 'FALLBACK'; variant: string; onError: (code: string) => void }
 - In FALLBACK mostra una timeline testuale delle eliminazioni già ricevute.
- **ResultsPanel** { positions: number[]; onVerify: () => void }
 - Click onVerify chiama getProvablyFair e apre modale.
- **ErrorBanner** { message: UXMessageKey | null } (legge mapping §5.1)

A2) Azioni Zustand (obbligatorie)

- bets.store.markPending(item) → push in pending e rimuove da cart.
- bets.store.markConfirmed(betId) → sposta da pending a confirmed.
- bets.store.markSettled(betId, result, payout) → sposta in settled.
- match.store.setPhase(phase, msRemaining) → aggiorna timer e fase.
- match.store.setLastEventId(evtId) → usato dal WS per resync.
- user.store.setSessionStatus('INVALID' | 'EXPIRED') → forza logout soft.

A3) Servizio REST (client.ts) — Interceptors vincolanti

- **Auth:** legge `token` da `user.store`. Se mancante → non inviare la chiamata e dispatch `UX_SESSION_INVALID`.
- **Correlation:** genera un nuovo `correlationId` per ogni azione utente; propaga su log UI e su chiamata.
- **Idempotency:** solo per `placeBet`; genera chiave una volta per click; se l'utente riclicca entro 2 minuti, riusa la stessa chiave.
- **Error:** se `error.code` appartiene a categoria AUTH/GEO → `user.store.setSessionStatus('INVALID')` e mostra `UX_SESSION_INVALID`. NET/DB/RGS → `UX_SERVICE_BUSY`. Video → `UX_CONTENT_UNAVAILABLE`.

A4) Servizio WS (wsGateway.ts) — Stato macchina client

Stati: `DISCONNECTED` → `CONNECTING` → `JOINED` → `RECONNECTING/BACKOFF` → `JOINED`.

- **CONNECTING:** apre socket, invia `room:join{roomId,token,correlationId}`; queue locale per messaggi fino ad ack.
- **JOINED:** sblocca queue. Aggiorna `room.store.status='JOINED'`.
- **RECONNECTING:** tenta connessione, all'open invia `sync:state{lastEventId}`; su `sync:ack` ripristina `JOINED`.
- **Errori:** WS invalid message/protocol → mostra `UX_CONN_ISSUE` e chiude.

Handler evento → azione vincolante:

- `match:phase` → `match.store.setPhase(phase, msRemaining)`.
- `match:sequence_hash` → `match.store.setServerSeedHash(hash)`.
- `match:elimination` → aggiorna UI di VideoBoard (o fallback) e timeline.
- `match:results` → `match.store.setResults(positions, serverSeed)`.
- `bet:accepted` → `bets.store.markConfirmed(betId)`.
- `bet:rejected` → mapping RW→UX (toast locale dettagliato se input non valido) + `bets.store.lastError`.
- `bet:settled` → `bets.store.markSettled(...)` e `user.store.setBalance(newBalance?)` se incluso.

A5) Flussi UX vincolanti

- **On load:** `/assign` → `connect()` → `/match/current`.
- **Place bet (WS first, REST fallback):** se WS OK usa `bet:place`; altrimenti POST `/bets/place`.
- **Reality check:** blocca `onPlace` fino ad ACK; “continua” sblocca immediatamente, “esci” chiude WS e naviga alla `HelpPage`.
- **Session invalid:** chiude WS, pulisce store, reindirizza alla pagina di login dell’operatore.

A6) Video mapping vincolante

- Naming clip: `{roomId}/{matchId}/{variant}.mp4` o HLS equivalente. `variant` derivato da `sequence` e `killer/victim`.
- Prefetch durante BETTING; se 404 → `RW-VID-001` → `UX_CONTENT_UNAVAILABLE` e passa a fallback testuale.

A7) Performance e sicurezza (checklist)

- Bundle iniziale < 300KB gzip; `VideoBoard` lazy.
- Nessun `innerHTML`; CSP attiva; disabilitata per-message WS compression.
- Token solo in memoria; clear all store su `SESSION_INVALID`.

A8) Test richiesti (FE)

- Unit: store actions, mapping errori, idempotency manager.
- Contract: con OpenAPI/AsyncAPI (schema `shared-types`).
- E2E: drop WS in PLAYING, resync corretta; reality check; video 404 → fallback.

7) Orchestrator (Node/TS)

7.1 API

- POST `/rooms` → crea stanza { `roomId`, `wsEndpoint`, `maxPlayers` }
- DELETE `/rooms/{roomId}` → drain & terminate
- POST `/assign` → { `playerId`, `jurisdiction` } → { `roomId`, `wsEndpoint` }
- GET `/rooms` → lista/health

7.2 Algoritmi

- **Assignment:** stanza ACTIVE con `playerCount < 0.9*max`; minimizza `|count-2500|`; rispetta `SINGLE_ACTIVE_SESSION`.
- **Jitter:** offset uniforme $\pm \text{ROUND_CLOSE_JITTER_MS}$; prova riduzione spike $\geq 60\%$.

7.3 Catalogo errori orchestrator

- RW-ORCH-001 ASSIGNMENT_FAILED, RW-ORCH-002 ROOM_NOT_FOUND, RW-ORCH-003 ROOM_AT_CAPACITY, RW-ORCH-004 HEALTH_STALE, RW-ORCH-006 REGION_NOT_SUPPORTED.

8) Backend del Gioco (NestJS/TS)

8.1 State machine round (per stanza)

WAITING → BETTING(60s) → LOCK → PLAYING(14–20s) → RESULTS(7s) → SETTLEMENT → NEXT

8.2 Provably Fair (dettaglio implementativo)

- Pre-bet: genera `serverSeed` (32 byte), persiste `serverSeedHash = SHA256(serverSeed)`.
- ClientSeed: hash cumulativo deterministico delle bet della stanza (`SHA256(JSON.stringify(sortedBets))`).
- CombinedSeed: `SHA256(serverSeed || clientSeed || roomId || closeTsISO)` (concatenazione **byte** con delimiter `0x1F`).
- RNG: `HMAC_DRBG(combinedSeed) → shuffle [1..8]` con Fisher–Yates.
- Post-match: rivela `serverSeed`; API di verifica restituisce `serverSeed`, `clientSeedHash`, `sequence` e guida di verifica.

8.3 Bet processing & settlement

- Intake: PENDING fino a validazione RGS (batch a T-0); idempotency su `Idempotency-Key`.
- Close: lock atomico stanza (`Redis SETNX` chiave `lock:room:{roomId}:match:{matchId}` TTL 60s).
- Settlement: calcolo vincite; outbox eventi CREDIT batch; retry con backoff; rollback ACID on failure; riconciliazione.

8.4 Persistenza (DDL essenziale in /spec/sql/backend-gioco.sql)

- `matches, bets, settlements, audit_logs`; indici: `bets(matchId)`, `bets(playerId)`, `settlements(matchId)`.

8.5 Errori backend gioco

- `RW-GAME-001...020` (vedi tassonomia).

8.A) Backend del Gioco — Addendum VINCOLANTE (sostituisce e dettaglia §§8.1–8.16)

Nota: questa sezione è **prescrittiva** e **sostituisce** i sottoparagrafi di §8 dove più dettagliata.

B1) Grafo moduli e dipendenze (fisso)

- `bets.controller` → `bets.service` → `unitOfWork` → `bets.repository` | `rgs.client` | `match.repository` | `outbox`.
- `match.controller` → `match.service` → `match.repository` | `cache`.
- `room.gateway` → `match.service` | `bets.service` (solo orchestrazione eventi, niente logica di business nelle classi WS).
- `settlement.service` → `outbox` → `rgs.client` → aggiorna `bets.repository`.

B2) Pipeline **POST /bets/place** (step-by-step vincolante)

1. **Auth/Geo** (guard): se KO → `RW-AUTH-*/RW-GEO-002`.
2. **ZodValidationPipe** su body.
3. **Rate limit per sessione**: 1 richiesta ogni 2s → `RW-NET-007` se violata.
4. **UnitOfWork.begin()**.
5. **Read match**: se fase ≠ `BETTING` → `RW-GAME-001`.
6. **Idempotency**: cerca `idempotencyKey`; se esiste → restituisce 201 idempotente con stesso `betId`.
7. **Business validation**: tipo/selection, stake nei limiti (in base a `limits` RGS/giurisdizione).
8. **Persist PENDING** (`bets`), audit log.
9. **UnitOfWork.commit()**.
10. WS `bet:accepted` verso il client.

(Il **debit** finanziario è delegato a RGS e può avvenire in batch a T-0; visualizzazione è separata dal regolamento)

B3) Lock & state machine (stanza)

- Una sola istanza di `match.scheduler` per `roomId`. Lock Redis con chiave `lock:room:{roomId}:match:{matchId}`.
- Il lock dura fino al completamento di `LOCK→PLAYING`.

B4) Provably Fair (dettagli implementativi)

- `serverSeed` 32 byte generati al round start; hash salvato e pubblicato.
- `clientSeedHash` derivato dalle bet aggregate della stanza al momento del lock.
- `combinedSeed` = hash dei dati precedenti + `roomId` + `closeTsISO`.
- CSPRNG HMAC_DRBG; shuffle deterministica con Fisher-Yates; `sequence` immutabile.
- Endpoint di verifica restituisce materiale per rifare i passaggi lato client.

B5) Outbox/Inbox e job BullMQ (nomi e payload)

- Queues: `debitBatch`, `creditBatch`, `reconcile`.
- Payload `creditBatch` item: { `playerId`, `amount`, `currency:'EUR'`, `roundId`, `idempotencyKey`, `correlationId` }.
- Concorrenza: 8 worker per coda per pod; backoff expo con jitter; DLQ se esauriti tentativi.

B6) WS Gateway (regole di sicurezza e flusso)

- Namespace `/ws/:roomId`; verifica `subprotocol==='rawwar.v1'`; origin check; max payload 64k; ping/pong 30s.
- Messaggi ammessi solo quelli elencati in §4.3; qualsiasi altro → `error{code:'RW-NET-008'}` e chiusura.
- Su `sync:state{lastEventId}`: invia delta eventi e poi `sync:ack`.

B7) Settlement & RGS (vincoli)

- `onRoundClose` calcola esiti e crea batch `CREDIT` (≤ 200).
- Idempotenza RGS: header `Idempotency-Key` per ogni item; riprovare solo su `RW-RGS-008` (timeout) e `RW-NET-009` (gateway busy).
- Al successo RGS: emit `bet:settled` a ciascun client coinvolto.

B8) Error handling server (bloccante)

- **REST:** solo codici di `REST_ERROR_REGISTRY`; non ammessi altri; envelope standard.
- **WS:** `error` solo con codici di `WS_ERROR_REGISTRY` relativi al messaggio che l'ha causato.
- **Fallback:** eccezioni non classificate → `RW-SYS-000` + log critico con `correlationId`.

B9) Metriche minime

- `http_request_duration_seconds` per endpoint, `ws_clients_gauge{roomId}`, `rounds_total{phase}`, `bets_placed_total`, `rgs_credit_timeouts_total`.

B10) Test minimi BE

- Unit: `bets.service` (idempotenza, limiti), `match.scheduler` (transizioni), `provablyfair.service` (ripetibilità), `rgs.client` (retry/breaker).
- Integration: `outbox`→`BullMQ`→`mock` RGS con Postgres/SQLite reali.
- Contract: allineamento OpenAPI/AsyncAPI.

9) RGS (Mock realistico)

9.1 API (batch)

- `/session/validate`, `/round/start`, `/transaction/debit`, `/transaction/credit`, `/round/end`.

9.2 Regole

- Idempotency 48h; Two-Phase Commit opzionale; Reconciliation hourly+daily.
- Errori: `RW-RGS-001...015` (inclusi `AML 014/015`, `partial batch 009`).

10) Casino (Mock)

- Wallet/limits/kyc/exclusions; webhooks `playerExcluded`, `balanceChanged`; failure model realistico: 0.1–0.5% error, 0.1% timeout.

11) Video pipeline

- ABR ladder: 1080p/720p; preload durante `BETTING`; fallback testuale su `RW-VID-*`.

- CDN: signed URLs (TTL 5 min); header `Cache-Control: public, max-age=60`; invalidazione edge su 404 multiple.
- Manifests: naming deterministico `/ {roomId} / {matchId} / {variant} .m3u8`.

12) Sicurezza

- **Auth:** JWT RS256 tra servizi; exp 15m; rotation; aud/iss stretti; `SINGLE_ACTIVE_SESSION` enforced.
- **Transport:** TLS 1.3; HSTS; **mTLS intra-cluster** (Prod).
- **Frontend:** CSP strict + **SRI obbligatorio** per tutti gli script; nessun inline senza SRI; violazione = errore CI.
- **WS hardening:** origin check, subprotocol whitelist, **max payload 64KB**, **no per-message compression**, ping/pong 30s.
- **Secret mgmt:** segreti in KMS/HSM; rotazione chiavi trimestrale; audit accessi.
- **OWASP mapping:** XSS → CSP+SRI; CSRF → token/JWT header; SSRF → allowlist; IDOR → authZ per risorsa; RCE → no eval, SCA.

13) Compliance & Legal

- **GLI-19:** RNG CSPRNG, display RTP/paytable, recovery/incomplete game, audit immutabile; network logging 24/7 entry/exit.
- **Responsible Gaming:** reality check configurabile + ACK obbligatorio; self-exclusion immediate upon reception; cooling-off aumento limiti.
- **Geo/VPN:** Casino primary + Game fail-safe (IP+headers) su `/assign` e pre-round; errori `RW-GEO-001/002`.
- **AML:** retention min 5 anni su record AML/ledger; trigger risk-based (`RW-RGS-014/015`); manual review `RW-CAS-009`.
- **GDPR:** anonimizzazione post-retention; diritto oblio compatibile con AML/audit (esente fino scadenza).
- **Disclosure UI:** Help screen con **RTP 95.50%** + link certificato lab, paytable, min/max bet, maxWin, guida provably fair.

14) Performance, DR e scaling

- Target: 200 stanze × 2,500 utenti = **500k CCU**.
- RGS: picco T-0 ≤ **5k TPS** con batching ≤ 200:1 + jitter ±2s.

- Autoscaling: HPA su CPU + queue depth + WS conns; spawn stanze via Orchestrator.
- DB: partition per matchId; indici selettivi; pgBouncer.
- **RPO/RTO**: $RPO \leq 5m$, $RTO \leq 15m$; test DR mensili (restore + failover).

Load test plan (k6/Locust):

- Scenario spike T-0 (~1M bet/s fleet-wide, ~5k TPS verso RGS) → p95 debit/credit < 200ms; error rate < 1%.
- Scenari WS drop 10% + video 404 1% → UX facade corretta e resilienza.

15) Observability & Audit

- **Logs**: pino JSON; `correlationId` obbligatorio; PII masked; livelli `info` | `warn` | `error`.
- **Metrics**: Prometheus (`latency p50/p95/p99`, TPS, queue depth, ws conns, error rates per RW-code, video timeouts, `ws_reconnect_attempts_total{roomId}`, `ws_resync_total{roomId}`).
- **Traces**: OpenTelemetry HTTP+WS; baggage con `roomId/matchId`.
- **Audit**: transazioni e settlement immutabili; **comportamento saturazione log**: offload su storage remoto; **hard-stop** se auditabilità non garantita.

Alert predefiniti:

- `rgs_debit_timeout_ratio > 0.05 for 5m`
- `settlement_failed_jobs > 0 for 1m`
- `ws_drop_rate > 0.1 in 1m`
- `queue_depth > 10000 for 2m`
- `video_404_rate > 0.005 in 5m`

16) Deployment & CI/CD

- **Docker**: multi-stage per app; immagini firmate; SBOM generato.
- **Pre-Prod**: docker-compose (SQLite, Redis single, 10 rooms, 56 video).
- **Prod**: Kubernetes (Helm): Deployments, HPA, PDB, NetworkPolicy, Ingress TLS, Secrets, Operators (Postgres HA, Redis Cluster).

- **CI/CD:** lint → unit → contract → integration → build → sbom/sca → sign → deploy blue/green; gates su test & coverage.
- **Feature flags:** rollout canary (1%, 10%, 50%, 100%).

17) Testing & Quality gates (bloccanti)

- **Unit:** RNG/shuffle, settlement, DTO validators.
- **Contract:** OpenAPI/AsyncAPI + Pact; **NO deploy** se falliscono.
- **Integration:** orchestrator↔backend, backend↔RGS, RGS↔casino.
- **E2E:** 10×100 in Pre-Prod; smoke ad ogni merge.
- **Load:** 200 rooms; spike T-0; report p50/p95/p99; error rate <1%.
- **Fault-injection:** kill settler, rallenta RGS, drop WS 50%, CDN 404.
- **Coverage** ≥ 80% , ESLint/Prettier on; **No TODO/TBD**; SCA clean (no High/Critical).

18) Paytable & Regole

- **RTP 95.50%** (certificato). Odds fisse:
 - WINNER 7.64
 - FTD 7.64
 - TOP2 27
 - TOP3 53
 - EXACT2 55
 - EXACT3 335
- **Payout:** `stake × odds`; cap **maxWin** per giurisdizione (default 3000.00).
- Help screen: payable, min/max bet, maxWin, **link certificato RTP**, guida provably fair.

19) Admin & Backoffice (API interne)

- `POST /admin/v1/rooms/{roomId}/drain` → inibisce nuovi join, chiude dopo round.
- `POST /admin/v1/matches/{matchId}/void` → VOID round conforme policy (log audit obbligatorio).

- GET /admin/v1/health → stato servizi.

20) Checklists bloccanti

- Contratti OpenAPI/AsyncAPI generati in /spec e versionati.
- Mapping RW-code → Facade UX implementato in /packages/shared-lib/errors/facade.ts.
- Policy CSP applicata e testata; no inline scripts senza SRI.
- Idempotency verificata (replay test) su debit/credit.
- Outbox/Inbox con DLQ e retry con backoff.
- Load test passato (TPS, p95, error rate) e report archiviato.
- DR test (RPO/RTO) eseguito e runbook aggiornato.
- CI Gates (§29) tutti verdi.

21) Appendici (schemi) — estratti vincolanti

21.1 JSON Schema — POST /api/v1/bets/place

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["matchId", "type", "selection", "stake"],
  "properties": {
    "matchId": { "type": "string", "minLength": 3 },
    "type": { "enum":
["WINNER", "FTD", "TOP2", "TOP3", "EXACT2", "EXACT3"] },
    "selection": { "type": "array", "items": { "type":
"integer", "minimum": 1, "maximum": 8 }, "minItems": 1,
"maxItems": 3, "uniqueItems": true },
    "stake": { "type": "string", "pattern": "^\\d+\\.\\d{2}$"
  }
}
```

21.2 JSON Schema — Envelope errore REST

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
```

```

"required": ["error"],
"properties": {
  "error": {
    "type": "object",
    "required": ["code", "message"],
    "properties": {
      "code": { "type": "string", "pattern": "^RW-[A-Z]+-\\d{3}$" },
      "message": { "type": "string" },
      "correlationId": { "type": "string" }
    }
  }
}
}

```

21.3 AsyncAPI — events (estratto testuale)

```

channel: ws/{roomId}
  messages:
    room:joined: { roomId }
    room:playerCount: { count }
    match:phase: { phase, msRemaining }
    match:sequence_hash: { serverSeedHash }
    match:elimination: { index, playerId }
    match:results: { positions[8], serverSeed }
    bet:accepted: { betId, status }
    bet:rejected: { betId, reason }
    bet:canceled: { betId, status }
    bet:settled: { betId, result, payout }
    balance:updated: { balance }
    error: { code, message, correlationId }
    sync:state (client→server): { lastEventId }
    sync:ack (server→client): { fromEventId, toEventId }

```

Vincolo finale: Endpoint, eventi WS, **codici errore RW-***, **messaggi UX facade**, campi e schemi sono **vincolanti**. Ogni deviazione è un **errore bloccante** e deve fallire la CI (contract tests).

22) Matrici di Gestione Errori (ESAUSTIVE E VINCOLANTI)

Questa sezione elenca **tutti gli error handler richiesti** per **ogni singola interazione**: endpoint REST, messaggi WS, job asincroni, video, startup/config, admin. Ogni voce specifica **Codici RW-*** ammessi, **facade UX** mostrato, **azioni FE** obbligatorie, **logging/metrics** e **retry/backoff**.

22.0 Regole comuni (obbligatorie)

- **Solo codici dal catalogo (§5.2 + Appendice D — Registry Errori).** Qualsiasi altro codice = errore CI.
- **Envelope REST** come in §4.2; ****evento WS `error`**** come in §4.3.
- **UX shown:** solo i 6 messaggi (§5.1). Mapping in `/packages/shared-lib/errors/facade.ts`.
- **Log:** livello `warn|error` con `code, correlationId, roomId, matchId, playerId?` (pseudonimizzato), durata richiesta.
- **Metrics:** `counter errors_total{code=..., endpoint=...}` e `histogram request_duration_seconds{...}`.
- **Retry policy:** solo dove indicato (backoff esponenziale jittered). Nessun retry infinito.

22.1 Orchestrator — REST

POST /assign

- **Errori:** RW-ORCH-001, RW-ORCH-002, RW-ORCH-003, RW-ORCH-006, RW-GEO-002, RW-AUTH-001/002/003, RW-DB-003, RW-SYS-000
- **UX:** ASSIGNMENT_FAILED/ROOM_AT_CAPACITY/REGION_NOT_SUPPORTED/GEO_BLOCKED/AUTH_* → UX_SESSION_INVALID o UX_CONN_ISSUE (per RW-DB-003), geo → UX_SESSION_INVALID.
- **FE:** se UX_SESSION_INVALID ⇒ logout soft + prompt login; se ROOM_AT_CAPACITY ⇒ ritenta tra 2s.
- **Retry:** sì per RW-ORCH-001 (3 tentativi: 200ms, 500ms, 1s); no per geo/auth.

POST /rooms, DELETE /rooms/{roomId}, GET /rooms

- **Errori:** RW-ORCH-002/004, RW-DB-003, RW-CFG-001, RW-SYS-000
- **UX:** admin-only (non esposto al player). FE backoffice mostra banner UX_SERVICE_BUSY.

22.2 Backend del Gioco — REST

GET /api/v1/match/current

- **Errori:** RW-GAME-018, RW-AUTH-001/002/003, RW-GEO-002, RW-DB-003, RW-SYS-000

- **UX:** ROUND_NOT_FOUND → UX_SERVICE_BUSY; AUTH/GEO → UX_SESSION_INVALID.

POST /api/v1/bets/place

- **Errori ammessi:** vedi REST_ERROR_REGISTRY['POST /api/v1/bets/place'] in §23.1.
- **UX:**
 - INSUFFICIENT_FUNDS → UX_INSUFFICIENT_FUNDS
 - LIMIT_EXCEEDED | EXCLUSION_ACTIVE | RG → UX_LIMITS
 - MATCH_CLOSED | PHASE_TRANSITION_CONFLICT | INVALID_SELECTION | BET_TOO_* | MAX_WIN_EXCEEDED → UX_SERVICE_BUSY (toast esplicativo locale)
 - RGS_TIMEOUT | GATEWAY_OVERLOADED | DB_* → UX_SERVICE_BUSY
 - AUTH/GEO → UX_SESSION_INVALID
- **FE:** disabilita bottone 1s su RATE_LIMITED; scroll a Help se INVALID_SELECTION.
- **Retry:** automatico **NO**; suggerire retry manuale per RW-NET-009, non per errori semantici.

DELETE /api/v1/bets/{betId}

- **Errori:** RW-GAME-019/001/018, RW-AUTH-002, RW-DB-003, RW-SYS-000
- **UX:** UX_SERVICE_BUSY oppure UX_SESSION_INVALID.

GET /api/v1/history/recent

- **Errori:** RW-AUTH-002, RW-DB-003, RW-SYS-000
- **UX:** UX_SERVICE_BUSY o UX_SESSION_INVALID.

GET /api/v1/paytable

- **Errori:** RW-CFG-001, RW-DB-004, RW-DB-003, RW-SYS-000
- **UX:** UX_CONTENT_UNAVAILABLE.

GET /api/v1/provably-fair/{matchId}

- **Errori:** RW-GAME-018, RW-GAME-013, RW-DB-003, RW-SYS-000
- **UX:** UX_CONTENT_UNAVAILABLE.

22.3 WebSocket — Client→Server

room:**join**

- **Errori:** RW-AUTH-001/002/003, RW-GEO-002, RW-GAME-020, RW-NET-008, RW-DB-003, RW-SYS-000
- **UX:** UX_SESSION_INVALID (auth/geo/version), UX_CONN_ISSUE (rete/db).
- **FE:** mostra banner persistente + auto-reconnect backoff.

bet:**place**

- **Errori:** stessi di POST /bets/place + RW-NET-007 (rate limit WS), RW-NET-006/008 (payload/protocollo)
- **UX:** come REST; WS-specific → UX_CONN_ISSUE.

bet:**cancel**

- **Errori:** RW-GAME-019/001/018, RW-AUTH-002, RW-NET-006/008, RW-DB-003, RW-SYS-000
- **UX:** UX_SERVICE_BUSY o UX_SESSION_INVALID o UX_CONN_ISSUE.

sync:**state**

- **Errori:** RW-NET-006 (too large), RW-NET-008 (protocol), RW-GAME-018 (match), RW-DB-003, RW-SYS-000
- **UX:** UX_CONN_ISSUE o UX_SERVICE_BUSY.

heartbeat

- **Errori:** RW-NET-005 (timeout), RW-NET-004 (close abnormal)
- **UX:** UX_CONN_ISSUE.

22.4 WebSocket — Server→Client

- **bet:rejected.reason** può essere: tutti gli errori dell'endpoint **bets/place** (§22.2) + RW-NET-007.
- Evento **error** può veicolare: RW-AUTH-*, RW-GEO-002, RW-NET-*, RW-GAME-011/015/020, RW-DB-003, RW-SYS-000.
- Gli altri eventi non generano errori, ma se la sessione diventa invalida il server invia **error** e chiude.

22.5 Backend del Gioco ↔ RGS

/session/validate: RW-RGS-003 (session), RW-AUTH-* (token inter-svc), RW-RGS-008 (timeout), RW-DB-003, RW-SYS-000

/round/start: RW-RGS-010 (non aperto), RW-RGS-008, RW-DB-003, RW-SYS-000

/transaction/debit (batch): RW-RGS-001/002/003/005/006/007/008/009, RW-DB-003, RW-SYS-000

/transaction/credit (batch): RW-RGS-008/009/013, RW-DB-003, RW-SYS-000

/round/end: RW-RGS-011, RW-RGS-008, RW-DB-003, RW-SYS-000

22.6 RGS ↔ Casino

/wallet/debit|credit: RW-CAS-001/002/006/008/010, RW-RGS-008 (timeout), RW-SYS-000

/limits/check: RW-CAS-003, RW-RGS-008, RW-SYS-000

Webhooks (playerExcluded, balanceChanged): RW-CAS-007 (delivery), RW-NET-009, RW-SYS-000

22.7 Job asincroni (BullMQ)

debitBatch: RW-RGS-008/009, RW-DB-003, RW-Q-003, RW-SYS-000

creditBatch/settlement: RW-RGS-008/009/013, RW-GAME-014, RW-DB-003, RW-Q-003, RW-SYS-000

compaction/audit-offload: RW-DB-003, RW-Q-002, RW-Q-003, RW-SYS-000

22.8 Video pipeline

prefetchManifest: RW-VID-001/002/006/008, RW-NET-001, RW-SYS-000

playClip: RW-VID-003/004/005/007, RW-NET-009, RW-SYS-000

abrSwitch: RW-VID-007, RW-NET-009

UX: tutti mappano a UX_CONTENT_UNAVAILABLE o UX_CONN_ISSUE (banner + fallback testo se FEATURE_FLAG_VIDEO_FALLBACK_TEXT=true).

22.9 Startup & Config (tutti i servizi)

loadEnv: RW-CFG-001, RW-CFG-002, RW-SYS-000

connectDB: RW-DB-003, RW-DB-005, RW-SYS-000

migrateDB: RW-DB-005, RW-DB-002, RW-SYS-000

connectRedis: RW-CACHE-001,RW-SYS-000

tls/bootstrap: RW-SEC-002,RW-SYS-000

22.10 Admin & Backoffice

POST /admin/v1/rooms/{roomId}/drain: RW-ORCH-002,RW-DB-003,RW-SYS-000

POST /admin/v1/matches/{matchId}/void: RW-GAME-018,RW-GAME-011,RW-DB-003,
RW-SYS-000

GET /admin/v1/health: RW-DB-003,RW-SYS-000

23) Implementazione Handler — Specifiche vincolanti

23.1 Registry dei codici per endpoint/evento (bloccante)

File: /packages/shared-types/errors/registry.ts

```
export const REST_ERROR_REGISTRY: Record<string,  
ReadonlyArray<string>> = {  
  'POST /assign': ['RW-ORCH-001','RW-ORCH-002','RW-  
ORCH-003','RW-ORCH-006','RW-GEO-002','RW-AUTH-001','RW-  
AUTH-002','RW-AUTH-003','RW-DB-003','RW-SYS-000'],  
  'GET /api/v1/match/current': ['RW-GAME-018','RW-  
AUTH-001','RW-AUTH-002','RW-AUTH-003','RW-GEO-002','RW-  
DB-003','RW-SYS-000'],  
  'POST /api/v1/bets/place': ['RW-GAME-001','RW-  
GAME-002','RW-GAME-003','RW-GAME-004','RW-GAME-005','RW-  
GAME-006','RW-GAME-007','RW-GAME-008','RW-GAME-009','RW-  
GAME-010','RW-GAME-011','RW-GAME-017','RW-GAME-018','RW-  
RGS-001','RW-RGS-002','RW-RGS-003','RW-NET-009','RW-  
DB-003','RW-SYS-000'],  
  'DELETE /api/v1/bets/{betId}': ['RW-GAME-019','RW-  
GAME-001','RW-GAME-018','RW-AUTH-002','RW-DB-003','RW-  
SYS-000'],  
  'GET /api/v1/history/recent': ['RW-AUTH-002','RW-  
DB-003','RW-SYS-000'],  
  'GET /api/v1/paytable': ['RW-CFG-001','RW-DB-004','RW-  
DB-003','RW-SYS-000'],  
  'GET /api/v1/provably-fair/{matchId}': ['RW-GAME-018','RW-  
GAME-013','RW-DB-003','RW-SYS-000']  
};
```

```
export const WS_ERROR_REGISTRY: Record<string,
ReadonlyArray<string>> = {
  'room:join': ['RW-AUTH-001', 'RW-AUTH-002', 'RW-
AUTH-003', 'RW-GEO-002', 'RW-GAME-020', 'RW-NET-008', 'RW-
DB-003', 'RW-SYS-000'],
  'bet:place': ['RW-GAME-001', 'RW-GAME-002', 'RW-
GAME-003', 'RW-GAME-004', 'RW-GAME-005', 'RW-GAME-006', 'RW-
GAME-007', 'RW-GAME-008', 'RW-GAME-009', 'RW-GAME-010', 'RW-
GAME-011', 'RW-GAME-017', 'RW-GAME-018', 'RW-RGS-001', 'RW-
RGS-002', 'RW-RGS-003', 'RW-NET-006', 'RW-NET-007', 'RW-
NET-008', 'RW-DB-003', 'RW-SYS-000'],
  'bet:cancel': ['RW-GAME-019', 'RW-GAME-001', 'RW-
GAME-018', 'RW-AUTH-002', 'RW-NET-006', 'RW-NET-008', 'RW-
DB-003', 'RW-SYS-000'],
  'sync:state': ['RW-NET-006', 'RW-NET-008', 'RW-GAME-018', 'RW-
DB-003', 'RW-SYS-000'],
  'heartbeat': ['RW-NET-005', 'RW-NET-004']
};
```

CI rule: un test fallisce se un service lancia un codice **non** incluso nel proprio registry.

23.2 Backend del Gioco — ExceptionFilter (NestJS)

- File: `/apps/backend-gioco/src/common/filters/http-exception.filter.ts`
- Converte errori applicativi in envelope REST standard; allega `correlationId`; valida codice contro `REST_ERROR_REGISTRY`.

23.3 WebSocket — Error emitter

- File: `/apps/backend-gioco/src/ws/error-emitter.ts`
- Solo eventi `error` con codici da `WS_ERROR_REGISTRY[messageType]`.

23.4 Frontend — Error middleware

- File: `/apps/frontend/src/services/api/errorInterceptor.ts`; `/apps/frontend/src/services/ws/errorGateway.ts`.
- Mappa RW-code → Facade UX; side-effects per categoria (logout, banner, retry/backoff, fallback video).

23.5 Observability

- Metriche per codice (`errors_total{code}`), endpoint/evento, e mapping a UX (dashboard pronta in Grafana: `/docs/observability/dashboard.json`).

24) Fallback e Safe-guards

- RW-SYS-000 è l'unico fallback ammesso per eccezioni non classificate; **alert S1** se >0.5% negli ultimi 5m.
- Se un handler tenta di emettere un codice non registrato, lanciare RW-SYS-000 + log critico con `code_originale` e **bloccare il deploy** via CI.
- FE non deve mai mostrare un RW-* direttamente all'utente: solo i 6 messaggi facade (§5.1).

Appendice A — OpenAPI (canonico)

```
openapi: 3.1.0
info:
  title: RAWWAR REST API (canonico, vincolante)
  version: v1
servers:
  - url: https://api.example.com
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  parameters:
    Authorization:
      name: Authorization
      in: header
      required: true
      schema: { type: string, pattern: "^Bearer\\s.+" }
    XCorrelationId:
      name: X-Correlation-Id
      in: header
      required: true
      schema: { type: string, minLength: 3 }
    IdempotencyKey:
      name: Idempotency-Key
      in: header
      required: true
      schema: { type: string, minLength: 8 }
      description: "Richiesto SOLO dove indicato (finestra
48h)."
  schemas:
```

```

Money:
  type: string
  pattern: "^\\d+\\.\\d{2}$"
  examples: ["10.00", "0.20", "3000.00"]
ErrorEnvelope:
  type: object
  required: [error]
  properties:
    error:
      type: object
      required: [code, message]
      properties:
        code: { type: string, pattern: "^RW-[A-Z]+-\\d{3}$" }
        message: { type: string }
        correlationId: { type: string }
AssignRequest:
  type: object
  required: [playerId, jurisdiction]
  properties:
    playerId: { type: string }
    jurisdiction: { type: string }
AssignResponse:
  type: object
  required: [roomId, wsEndpoint]
  properties:
    roomId: { type: string }
    wsEndpoint: { type: string, format: uri }
MatchCurrentResponse:
  type: object
  required: [roomId, matchId, phase, msRemaining, serverSeedHash]
  properties:
    roomId: { type: string }
    matchId: { type: string }
    phase: { enum: ["BETTING", "PLAYING", "RESULTS"] }
    msRemaining: { type: integer, minimum: 0 }
    serverSeedHash: { type: string }
BetPlaceRequest:
  type: object
  required: [matchId, type, selection, stake]
  properties:
    matchId: { type: string }
    type: { enum: ["WINNER", "FTD", "TOP2", "TOP3", "EXACT2", "EXACT3"] }

```

```

    selection:
      type: array
      items: { type: integer, minimum: 1, maximum: 8 }
      minItems: 1
      maxItems: 3
      uniqueItems: true
    stake: { $ref: "#/components/schemas/Money" }
  BetAccepted:
    type: object
    required: [betId, status]
    properties:
      betId: { type: string }
      status: { enum: ["PENDING"] }
  HistoryItem:
    type: object
    required: [betId, matchId, type, selection, stake,
result, payout]
    properties:
      betId: { type: string }
      matchId: { type: string }
      type: { enum:
["WINNER", "FTD", "TOP2", "TOP3", "EXACT2", "EXACT3"] }
      selection:
        type: array
        items: { type: integer }
      stake: { $ref: "#/components/schemas/Money" }
      result: { enum: ["WIN", "LOSE"] }
      payout: { $ref: "#/components/schemas/Money" }
  PaytableResponse:
    type: object
    required: [rtp, odds, minBet, maxBet, maxWin]
    properties:
      rtp: { type: string, pattern: "^\\d+\\.\\d{2}$",
example: "95.50" }
      odds:
        type: object
        required: [WINNER, FTD, TOP2, TOP3, EXACT2, EXACT3]
        properties:
          WINNER: { type: string, example: "7.64" }
          FTD: { type: string, example: "7.64" }
          TOP2: { type: string, example: "27" }
          TOP3: { type: string, example: "53" }
          EXACT2: { type: string, example: "55" }
          EXACT3: { type: string, example: "335" }
        minBet: { $ref: "#/components/schemas/Money" }

```

```

        maxBet: { $ref: "#/components/schemas/Money" }
        maxWin: { $ref: "#/components/schemas/Money" }
    ProvablyFairResponse:
        type: object
        required: [serverSeedHash, serverSeed, clientSeedHash,
sequence]
        properties:
            serverSeedHash: { type: string }
            serverSeed: { type: string } # base64
            clientSeedHash: { type: string }
            sequence:
                type: array
                minItems: 8
                maxItems: 8
                items: { type: integer, minimum: 1, maximum: 8 }
            verifyUrl: { type: string, format: uri, nullable:
true }
    RgsBatchItem:
        type: object
        required: [playerId, amount, currency, roundId,
idempotencyKey]
        properties:
            playerId: { type: string }
            amount: { $ref: "#/components/schemas/Money" }
            currency: { type: string, enum: ["EUR"] }
            roundId: { type: string }
            idempotencyKey: { type: string }
    RgsBatchRequest:
        type: object
        required: [items]
        properties:
            items:
                type: array
                maxItems: 200
                items: { $ref: "#/components/schemas/
RgsBatchItem" }
    RgsBatchResult:
        type: object
        required: [status]
        properties:
            status: { enum: ["APPROVED", "REJECTED"] }
            reason: { type: string, pattern: "^RW-(RGS|CAS)-
\d{3}$", nullable: true }
            txId: { type: string, nullable: true }

```

```

        newBalance: { $ref: "#/components/schemas/Money",
nullable: true }
paths:
  /assign:
    post:
      summary: Assegna player a room
      security: [ { bearerAuth: [] } ]
      parameters:
        - $ref: "#/components/parameters/Authorization"
        - $ref: "#/components/parameters/XCorrelationId"
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/
AssignRequest" }
            examples:
              ok: { value: { playerId: "u_123", jurisdiction:
"MGA" } }
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema: { $ref: "#/components/schemas/
AssignResponse" }
              examples:
                ok: { value: { roomId: "r_42", wsEndpoint:
"wss://rooms.example/ws/r_42" } }
        "401": { description: AUTH KO, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        "403": { description: GEO KO, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        "409": { description: ROOM_AT_CAPACITY/
ASSIGNMENT_FAILED, content: { application/json: { schema:
{ $ref: "#/components/schemas/ErrorEnvelope" } } } }
        "503": { description: HEALTH_STALE/DB, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-ORCH-001", "RW-
ORCH-002", "RW-ORCH-003", "RW-ORCH-006", "RW-GEO-002", "RW-
AUTH-001", "RW-AUTH-002", "RW-AUTH-003", "RW-DB-003", "RW-
SYS-000"]

```

```

/rooms:
  get:
    summary: Lista/health rooms
    security: [ { bearerAuth: [] } ]
    parameters:
      - $ref: "#/components/parameters/Authorization"
      - $ref: "#/components/parameters/XCorrelationId"
    responses:
      "200":
        description: OK
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                required: [roomId, players, status]
                properties:
                  roomId: { type: string }
                  players: { type: integer }
                  status: { enum:
["CREATING", "ACTIVE", "DRAINING", "TERMINATED"] }
      "503": { description: Service busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-ORCH-004", "RW-DB-003", "RW-
CFG-001", "RW-SYS-000"]
    post:
      summary: Crea room (admin)
      security: [ { bearerAuth: [] } ]
      parameters: [ { $ref: "#/components/parameters/
Authorization" }, { $ref: "#/components/parameters/
XCorrelationId" } ]
      responses:
        "201": { description: Created }
        "503": { description: Service busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-DB-003", "RW-CFG-001", "RW-
SYS-000"]
  /rooms/{roomId}:
    delete:
      summary: Drain & terminate room (admin)
      security: [ { bearerAuth: [] } ]
      parameters:

```

```

- $ref: "#/components/parameters/Authorization"
- $ref: "#/components/parameters/XCorrelationId"
- name: roomId
  in: path
  required: true
  schema: { type: string }
responses:
  "202": { description: Accepted }
  "404": { description: ROOM_NOT_FOUND, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
  "503": { description: Service busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
  x-allowed-error-codes: ["RW-ORCH-002", "RW-DB-003", "RW-
SYS-000"]
/api/v1/match/current:
  get:
    summary: Stato corrente match per room
    security: [ { bearerAuth: [] } ]
    parameters: [ { $ref: "#/components/parameters/
Authorization" }, { $ref: "#/components/parameters/
XCorrelationId" } ]
    responses:
      "200":
        description: OK
        content:
          application/json:
            schema: { $ref: "#/components/schemas/
MatchCurrentResponse" }
      "401": { description: AUTH KO, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      "403": { description: GEO KO, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      "404": { description: ROUND_NOT_FOUND, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      "503": { description: DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-GAME-018", "RW-
AUTH-001", "RW-AUTH-002", "RW-AUTH-003", "RW-GEO-002", "RW-
DB-003", "RW-SYS-000"]

```

```

/api/v1/bets/place:
  post:
    summary: Place bet (WS-first fallback)
    security: [ { bearerAuth: [] } ]
    parameters:
      - $ref: "#/components/parameters/Authorization"
      - $ref: "#/components/parameters/XCorrelationId"
      - $ref: "#/components/parameters/IdempotencyKey"
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: "#/components/schemas/BetPlaceRequest" }
          examples:
            ok: { value: { matchId: "m_1", type: "WINNER",
selection: [3], stake: "10.00" } }
    responses:
      "201": { description: Accepted, content:
{ application/json: { schema: { $ref: "#/components/schemas/BetAccepted" } } } }
      "400": { description: Validation /
INVALID_SELECTION / BET_TOO_* , content: { application/json:
{ schema: { $ref: "#/components/schemas/ErrorEnvelope" } } } }
      "401": { description: Auth, content: { application/
json: { schema: { $ref: "#/components/schemas/ErrorEnvelope" } } } }
      "403": { description: Geo/Limits/Exclusion (UX_LIMITS
or SESSION_INVALID), content: { application/json: { schema: {
$ref: "#/components/schemas/ErrorEnvelope" } } } }
      "409": { description: MATCH_CLOSED / PHASE_CONFLICT,
content: { application/json: { schema: { $ref: "#/components/
schemas/ErrorEnvelope" } } } }
      "422": { description: Business rules
(MAX_WIN_EXCEEDED, LIMIT_EXCEEDED), content: { application/
json: { schema: { $ref: "#/components/schemas/ErrorEnvelope" } } } }
      "429": { description: Rate limited (per session),
content: { application/json: { schema: { $ref: "#/components/
schemas/ErrorEnvelope" } } } }
      "503": { description: Service busy / Gateway
overloaded, content: { application/json: { schema: { $ref:
"#/components/schemas/ErrorEnvelope" } } } }

```

```

    "504": { description: RGS timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
    x-allowed-error-codes:
      - "RW-GAME-001", "RW-GAME-002", "RW-GAME-003", "RW-
GAME-004", "RW-GAME-005", "RW-GAME-006", "RW-GAME-007", "RW-
GAME-008", "RW-GAME-009", "RW-GAME-010", "RW-GAME-011", "RW-
GAME-017", "RW-GAME-018"
      - "RW-RGS-001", "RW-RGS-002", "RW-RGS-003", "RW-
NET-009", "RW-DB-003", "RW-SYS-000"
    /api/v1/bets/{betId}:
      delete:
        summary: Cancel bet (solo BETTING)
        security: [ { bearerAuth: [] } ]
        parameters:
          - $ref: "#/components/parameters/Authorization"
          - $ref: "#/components/parameters/XCorrelationId"
          - name: betId
            in: path
            required: true
            schema: { type: string }
        responses:
          "204": { description: Canceled }
          "401": { description: Auth, content: { application/
json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
          "404": { description: NOT_FOUND/ROUND_NOT_FOUND,
content: { application/json: { schema: { $ref: "#/components/
schemas/ErrorEnvelope" } } } }
          "409": { description: MATCH_CLOSED / PHASE_CONFLICT,
content: { application/json: { schema: { $ref: "#/components/
schemas/ErrorEnvelope" } } } }
          "503": { description: DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        x-allowed-error-codes: [ "RW-GAME-019", "RW-
GAME-001", "RW-GAME-018", "RW-AUTH-002", "RW-DB-003", "RW-
SYS-000" ]
    /api/v1/history/recent:
      get:
        summary: Storico recente bet dell'utente
        security: [ { bearerAuth: [] } ]
        parameters: [ { $ref: "#/components/parameters/
Authorization" }, { $ref: "#/components/parameters/
XCorrelationId" } ]

```

```

responses:
  "200":
    description: OK
    content:
      application/json:
        schema:
          type: array
          items: { $ref: "#/components/schemas/
HistoryItem" }
    "401": { description: Auth, content: { application/
json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
    "503": { description: DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
    x-allowed-error-codes: ["RW-AUTH-002", "RW-DB-003", "RW-
SYS-000"]
  /api/v1/paytable:
    get:
      summary: Paytable corrente
      parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema: { $ref: "#/components/schemas/
PaytableResponse" }
          "503": { description: CFG/DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
          x-allowed-error-codes: ["RW-CFG-001", "RW-DB-004", "RW-
DB-003", "RW-SYS-000"]
    /api/v1/provably-fair/{matchId}:
      get:
        summary: Materiale verifica provably fair
        parameters:
          - $ref: "#/components/parameters/XCorrelationId"
          - name: matchId
            in: path
            required: true
            schema: { type: string }
        responses:
          "200":

```

```

        description: OK
        content:
            application/json:
                schema: { $ref: "#/components/schemas/
ProvablyFairResponse" }
            "404": { description: ROUND_NOT_FOUND, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
            "422": { description: VERIFY_UNAVAILABLE (esito non
ancora pubblicabile), content: { application/json: { schema:
{ $ref: "#/components/schemas/ErrorEnvelope" } } } }
            "503": { description: DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        x-allowed-error-codes: ["RW-GAME-018", "RW-
GAME-013", "RW-DB-003", "RW-SYS-000"]
        /session/validate:
            post:
                summary: (RGS) Validazione sessione
                parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
                requestBody:
                    required: true
                    content:
                        application/json:
                            schema:
                                type: object
                                required: [playerId, casinoId, jurisdiction]
                                properties:
                                    playerId: { type: string }
                                    casinoId: { type: string }
                                    jurisdiction: { type: string }
                responses:
                    "200": { description: OK, content: { application/
json: { schema: { type: object, required: [status, limits,
currency], properties: { status: { enum: ["VALID"] }, limits:
{ type: object }, currency: { enum: ["EUR"] } } } } } }
                    "503": { description: timeout/DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
                x-allowed-error-codes: ["RW-RGS-003", "RW-RGS-008", "RW-
DB-003", "RW-SYS-000"]
        /round/start:
            post:
                summary: (RGS) Start round

```

```

    parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
    requestBody:
      required: true
      content:
        application/json:
          schema: { type: object, required: [roundId,
gameId, ts], properties: { roundId: { type: string }, gameId:
{ const: "RAWWAR" }, ts: { type: string, format: date-
time } } }
      responses:
        "200": { description: OK }
        "503": { description: timeout/DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        x-allowed-error-codes: [ "RW-RGS-010", "RW-RGS-008", "RW-
DB-003", "RW-SYS-000" ]
    /transaction/debit:
      post:
        summary: (RGS) Batch debit
        parameters: [ { $ref: "#/components/parameters/
XCorrelationId" }, { $ref: "#/components/parameters/
IdempotencyKey" } ]
        requestBody:
          required: true
          content:
            application/json:
              schema: { $ref: "#/components/schemas/
RgsBatchRequest" }
        responses:
          "200":
            description: Risultati per item
            content:
              application/json:
                schema:
                  type: object
                  required: [results]
                  properties:
                    results:
                      type: array
                      items: { $ref: "#/components/schemas/
RgsBatchResult" }
          "503": { description: GW busy/timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }

```

```

    "504": { description: timeout RGS, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
    x-allowed-error-codes: ["RW-RGS-001","RW-RGS-002","RW-
RGS-003","RW-RGS-005","RW-RGS-006","RW-RGS-007","RW-
RGS-008","RW-RGS-009","RW-DB-003","RW-SYS-000"]
    /transaction/credit:
    post:
        summary: (RGS) Batch credit
        parameters: [ { $ref: "#/components/parameters/
XCorrelationId" }, { $ref: "#/components/parameters/
IdempotencyKey" } ]
        requestBody:
            required: true
            content:
                application/json:
                    schema: { $ref: "#/components/schemas/
RgsBatchRequest" }
        responses:
            "200":
                description: Risultati per item
                content:
                    application/json:
                        schema:
                            type: object
                            required: [results]
                            properties:
                                results:
                                    type: array
                                    items: { $ref: "#/components/schemas/
RgsBatchResult" }
            "503": { description: GW busy/timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
            "504": { description: timeout RGS, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        x-allowed-error-codes: ["RW-RGS-008","RW-RGS-009","RW-
RGS-013","RW-DB-003","RW-SYS-000"]
    /round/end:
    post:
        summary: (RGS) End round
        parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
        requestBody:

```

```

    required: true
    content:
      application/json:
        schema: { type: object, required: [roundId,
result], properties: { roundId: { type: string }, result:
{ enum: ["OK"] } } }
      responses:
        "200": { description: OK }
        "503": { description: timeout/DB busy, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-RGS-011", "RW-RGS-008", "RW-
DB-003", "RW-SYS-000"]
    /wallet/debit:
      post:
        summary: (Casino mock) Debit singolo
        parameters: [ { $ref: "#/components/parameters/
XCorrelationId" }, { $ref: "#/components/parameters/
IdempotencyKey" } ]
        requestBody:
          required: true
          content:
            application/json:
              schema:
                type: object
                required: [playerId, amount, currency,
idempotencyKey]
                properties:
                  playerId: { type: string }
                  amount: { $ref: "#/components/schemas/
Money" }
                  currency: { enum: ["EUR"] }
                  idempotencyKey: { type: string }
        responses:
          "200": { description: OK (APPROVED/REJECTED),
content: { application/json: { schema: { type: object,
required: [status], properties: { status: { enum:
["APPROVED", "REJECTED"] }, reason: { type: string, nullable:
true } } } } } }
          "503": { description: GW busy/timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
          x-allowed-error-codes: ["RW-CAS-001", "RW-CAS-002", "RW-
CAS-006", "RW-CAS-008", "RW-CAS-010", "RW-RGS-008", "RW-SYS-000"]
    /wallet/credit:

```

```

    post:
      summary: (Casino mock) Credit singolo
      parameters: [ { $ref: "#/components/parameters/
XCorrelationId" }, { $ref: "#/components/parameters/
IdempotencyKey" } ]
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [playerId, amount, currency,
idempotencyKey]
              properties:
                playerId: { type: string }
                amount: { $ref: "#/components/schemas/
Money" }
                currency: { enum: ["EUR"] }
                idempotencyKey: { type: string }
      responses:
        "200": { description: OK (APPROVED/REJECTED) }
        "503": { description: GW busy/timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        x-allowed-error-codes: ["RW-CAS-001","RW-CAS-002","RW-
CAS-006","RW-CAS-008","RW-CAS-010","RW-RGS-008","RW-SYS-000"]
    /limits/check:
      post:
        summary: (Casino mock) Limits check
        parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
        requestBody:
          required: true
          content:
            application/json:
              schema: { type: object, required: [playerId,
stake], properties: { playerId: { type: string }, stake:
{ $ref: "#/components/schemas/Money" } } }
        responses:
          "200": { description: OK, content: { application/
json: { schema: { type: object, required: [ok], properties: {
ok: { type: boolean }, reason: { type: string, nullable: true
} } } } } }

```

```
    "503": { description: busy/timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
    x-allowed-error-codes: ["RW-CAS-003","RW-RGS-008","RW-
SYS-000"]
    /rgs/playerExcluded:
    post:
        summary: (Casino→RGS) Webhook esclusione
        parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
        requestBody:
            required: true
            content:
                application/json:
                    schema: { type: object, required: [playerId,
reason, ts], properties: { playerId: { type: string },
reason: { type: string }, ts: { type: string, format: date-
time } } }
        responses:
            "202": { description: Accepted }
            "503": { description: busy/timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        x-allowed-error-codes: ["RW-CAS-007","RW-NET-009","RW-
SYS-000"]
    /rgs/balanceChanged:
    post:
        summary: (Casino→RGS) Webhook saldo
        parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
        requestBody:
            required: true
            content:
                application/json:
                    schema: { type: object, required: [playerId,
newBalance, ts], properties: { playerId: { type: string },
newBalance: { $ref: "#/components/schemas/Money" }, ts:
{ type: string, format: date-time } } }
        responses:
            "202": { description: Accepted }
            "503": { description: busy/timeout, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
        x-allowed-error-codes: ["RW-CAS-007","RW-NET-009","RW-
SYS-000"]
```

```

/admin/v1/rooms/{roomId}/drain:
  post:
    summary: (Admin) Drain room
    security: [ { bearerAuth: [] } ]
    parameters:
      - $ref: "#/components/parameters/Authorization"
      - $ref: "#/components/parameters/XCorrelationId"
      - name: roomId
        in: path
        required: true
        schema: { type: string }
    responses:
      "202": { description: Accepted }
      "404": { description: ROOM_NOT_FOUND, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      "503": { description: busy/DB, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-ORCH-002", "RW-DB-003", "RW-
SYS-000"]
/admin/v1/matches/{matchId}/void:
  post:
    summary: (Admin) Void match
    security: [ { bearerAuth: [] } ]
    parameters:
      - $ref: "#/components/parameters/Authorization"
      - $ref: "#/components/parameters/XCorrelationId"
      - name: matchId
        in: path
        required: true
        schema: { type: string }
    responses:
      "202": { description: Accepted }
      "404": { description: ROUND_NOT_FOUND, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      "409": { description: STATE_CONFLICT, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      "503": { description: busy/DB, content:
{ application/json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-GAME-018", "RW-
GAME-011", "RW-DB-003", "RW-SYS-000"]

```

```

/admin/v1/health:
  get:
    summary: Health
    parameters: [ { $ref: "#/components/parameters/
XCorrelationId" } ]
    responses:
      "200": { description: OK }
      "503": { description: busy, content: { application/
json: { schema: { $ref: "#/components/schemas/
ErrorEnvelope" } } } }
      x-allowed-error-codes: ["RW-DB-003","RW-SYS-000"]

```

Appendice B — AsyncAPI (canonico)

```

asyncapi: 2.6.0
info:
  title: RAWWAR WS — room-scoped (canonico)
  version: v1
servers:
  rooms:
    url: wss://rooms.example.com/ws/{roomId}
    protocol: wss
    protocolVersion: "1.0"
    variables:
      roomId:
        description: roomId canonico (§0.1)
        default: r_42
  bindings:
    ws:
      headers:
        type: object
        properties:
          Sec-WebSocket-Protocol:
            type: string
            const: rawwar.v1
channels:
  ws/{roomId}:
    description: WebSocket room-scoped (subprotocol
rawwar.v1)
    bindings:
      ws:
        method: GET
        subscribe: # server -> client
        message:

```

```

    oneOf:
      - $ref: "#/components/messages/room_joined"
      - $ref: "#/components/messages/room_playerCount"
      - $ref: "#/components/messages/match_phase"
      - $ref: "#/components/messages/match_sequence_hash"
      - $ref: "#/components/messages/match_elimination"
      - $ref: "#/components/messages/match_results"
      - $ref: "#/components/messages/bet_accepted"
      - $ref: "#/components/messages/bet_rejected"
      - $ref: "#/components/messages/bet_canceled"
      - $ref: "#/components/messages/bet_settled"
      - $ref: "#/components/messages/balance_updated"
      - $ref: "#/components/messages/error"
      - $ref: "#/components/messages/sync_ack"
publish: # client -> server
  message:
    oneOf:
      - $ref: "#/components/messages/room_join"
      - $ref: "#/components/messages/bet_place"
      - $ref: "#/components/messages/bet_cancel"
      - $ref: "#/components/messages/heartbeat"
      - $ref: "#/components/messages/sync_state"
components:
  schemas:
    Money:
      type: string
      pattern: "^\\d+\\.\\d{2}$"
    Error:
      type: object
      required: [code, message, correlationId]
      properties:
        code: { type: string, pattern: "^RW-[A-Z]+-\\d{3}$" }
        message: { type: string }
        correlationId: { type: string }
    Positions:
      type: array
      minItems: 8
      maxItems: 8
      items: { type: integer, minimum: 1, maximum: 8 }
  messages:
    room_join:
      name: room:join
      payload:
        type: object
        required: [roomId, token, correlationId]

```

```

    properties:
      roomId: { type: string }
      token: { type: string }
      correlationId: { type: string }
  room_joined:
    name: room:joined
    payload:
      type: object
      required: [roomId]
      properties:
        roomId: { type: string }
  room_playerCount:
    name: room:playerCount
    payload: { type: object, required: [count], properties:
{ count: { type: integer, minimum: 0 } } }
  match_phase:
    name: match:phase
    payload:
      type: object
      required: [phase, msRemaining]
      properties:
        phase: { enum: ["BETTING","PLAYING","RESULTS"] }
        msRemaining: { type: integer, minimum: 0 }
  match_sequence_hash:
    name: match:sequence_hash
    payload: { type: object, required: [serverSeedHash],
properties: { serverSeedHash: { type: string } } }
  match_elimination:
    name: match:elimination
    payload:
      type: object
      required: [index, playerId]
      properties:
        index: { type: integer, minimum: 1, maximum: 7 }
        playerId: { type: integer, minimum: 1, maximum: 8 }
  match_results:
    name: match:results
    payload:
      type: object
      required: [positions, serverSeed]
      properties:
        positions: { $ref: "#/components/schemas/Positions"
}
        serverSeed: { type: string } # base64
  bet_accepted:

```

```

    name: bet:accepted
    payload: { type: object, required: [betId, status],
properties: { betId: { type: string }, status: { enum:
["PENDING"] } } }
    bet_rejected:
      name: bet:rejected
      payload: { type: object, required: [betId, reason],
properties: { betId: { type: string }, reason: { type:
string, pattern: "^RW-[A-Z]+-\d{3}$" } } }
    bet_canceled:
      name: bet:canceled
      payload: { type: object, required: [betId, status],
properties: { betId: { type: string }, status: { enum:
["CANCELED"] } } }
    bet_settled:
      name: bet:settled
      payload:
        type: object
        required: [betId, result, payout]
        properties:
          betId: { type: string }
          result: { enum: ["WIN", "LOSE"] }
          payout: { $ref: "#/components/schemas/Money" }
    balance_updated:
      name: balance:updated
      payload: { type: object, required: [balance],
properties: { balance: { $ref: "#/components/schemas/Money" }
} }
    error:
      name: error
      payload: { $ref: "#/components/schemas/Error" }
    heartbeat:
      name: heartbeat
      payload: { type: object, required: [ts], properties:
{ ts: { type: string, format: date-time } } }
    sync_state:
      name: sync:state
      payload: { type: object, required: [lastEventId],
properties: { lastEventId: { type: string } } }
    sync_ack:
      name: sync:ack
      payload: { type: object, required: [fromEventId,
toEventId], properties: { fromEventId: { type: string },
toEventId: { type: string } } }

```

Appendice C — RGS/Casino Schema (JSON Schema)

```
{
  "$id": "https://schemas.rawwar.example/rgs-casino.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "RGS & Casino canonical payloads",
  "type": "object",
  "properties": {
    "RgsSessionValidateRequest": {
      "type": "object",
      "required": ["playerId", "casinoId", "jurisdiction"],
      "properties": {
        "playerId": { "type": "string" },
        "casinoId": { "type": "string" },
        "jurisdiction": { "type": "string" }
      }
    },
    "RgsSessionValidateResponse": {
      "type": "object",
      "required": ["status", "limits", "currency"],
      "properties": {
        "status": { "enum": ["VALID"] },
        "limits": { "type": "object" },
        "currency": { "enum": ["EUR"] }
      }
    },
    "RgsRoundStartRequest": {
      "type": "object",
      "required": ["roundId", "gameId", "ts"],
      "properties": {
        "roundId": { "type": "string" },
        "gameId": { "const": "RAWWAR" },
        "ts": { "type": "string", "format": "date-time" }
      }
    },
    "RgsRoundEndRequest": {
      "type": "object",
      "required": ["roundId", "result"],
      "properties": {
        "roundId": { "type": "string" },
        "result": { "enum": ["OK"] }
      }
    },
    "RgsBatchItem": {
```

```

        "type": "object",
        "required":
["playerId","amount","currency","roundId","idempotencyKey"],
        "properties": {
            "playerId": { "type": "string" },
            "amount": { "type": "string", "pattern": "^\\d+\\.
\\d{2}$" },
            "currency": { "enum": ["EUR"] },
            "roundId": { "type": "string" },
            "idempotencyKey": { "type": "string" }
        }
    },
    "RgsBatchRequest": {
        "type": "object",
        "required": ["items"],
        "properties": {
            "items": {
                "type": "array",
                "maxItems": 200,
                "items": { "$ref": "#/properties/RgsBatchItem" }
            }
        },
        "description": "Finestra idempotency 48h (policy di
servizio).",
    },
    "RgsBatchResult": {
        "type": "object",
        "required": ["status"],
        "properties": {
            "status": { "enum": ["APPROVED","REJECTED"] },
            "reason": { "type": "string", "pattern": "^RW-(RGS|
CAS)-\\d{3}$", "nullable": true },
            "txId": { "type": "string", "nullable": true },
            "newBalance": { "type": "string", "pattern": "^\\d+\\.
\\d{2}$", "nullable": true }
        }
    },
    "CasinoDebitCreditRequest": {
        "type": "object",
        "required":
["playerId","amount","currency","idempotencyKey"],
        "properties": {
            "playerId": { "type": "string" },
            "amount": { "type": "string", "pattern": "^\\d+\\.
\\d{2}$" },

```

```

        "currency": { "enum": ["EUR"] },
        "idempotencyKey": { "type": "string" }
    }
},
"CasinoDebitCreditResponse": {
    "type": "object",
    "required": ["status"],
    "properties": {
        "status": { "enum": ["APPROVED", "REJECTED"] },
        "reason": { "type": "string", "pattern": "^RW-(CAS|
RGS)-\d{3}$", "nullable": true },
        "newBalance": { "type": "string", "pattern": "^\d+\.
\d{2}$", "nullable": true }
    }
},
"CasinoLimitsCheckRequest": {
    "type": "object",
    "required": ["playerId", "stake"],
    "properties": {
        "playerId": { "type": "string" },
        "stake": { "type": "string", "pattern": "^\d+\.
\d{2}$" }
    }
},
"CasinoLimitsCheckResponse": {
    "type": "object",
    "required": ["ok"],
    "properties": {
        "ok": { "type": "boolean" },
        "reason": { "type": "string", "pattern": "^RW-CAS-
\d{3}$", "nullable": true }
    }
},
"WebhookPlayerExcluded": {
    "type": "object",
    "required": ["playerId", "reason", "ts"],
    "properties": {
        "playerId": { "type": "string" },
        "reason": { "type": "string" },
        "ts": { "type": "string", "format": "date-time" }
    }
},
"WebhookBalanceChanged": {
    "type": "object",
    "required": ["playerId", "newBalance", "ts"],

```

```
    "properties": {
      "playerId": { "type": "string" },
      "newBalance": { "type": "string", "pattern": "^\\d+\\.\\d{2}$" },
      "ts": { "type": "string", "format": "date-time" }
    }
  },
  "additionalProperties": false
}
```

Appendice D — Registry Errori (vincolante)

room:join (WS C→S)	WS 1000/1008/1011/1 013	RW- AUTH-001,002 ,003 / RW- GEO-002 / RW-	SESSION_INVALID / CONN_ISSUE	backoff expo (0.5→30s)	
bet:place (WS)	WS 1000/1011/1013	come REST + RW-	come REST + CONN_ISSUE	no auto; suggerire retry manuale	
bet:cancel (WS C→S)	WS 1000/1011/1013	RW- GAME-019,001 ,018 / RW-	SERVICE_BUSY/ SESSION_INVALID	no	
sync:state (WS C→S)	WS 1000/1011/1013	RW- NET-006,008 / RW-	CONN_ISSUE / SERVICE_BUSY	backoff expo (0.5→30s)	
heartbeat (WS)	WS 1000/1006	RW- NET-005,004	CONN_ISSUE	backoff expo	
/session/ validate	200/503	RW-RGS-003 / RW-RGS-008 /	SERVICE_BUSY	sì (500/1000/2000ms)	
/round/ start	200/503	RW-RGS-010 / RW-RGS-008 /	SERVICE_BUSY	sì	
/transaction/debit (RGS)	200/503/504	RW- RGS-001,002,0 03,005,006,007, 008,009 / RW-	LIMITS/ INSUFFICIENT_FUNDS→UX corrispondente	sì su 503/504	
/transaction	200/503/504	RW- RGS-008,009,0	SERVICE_BUSY	sì su 503/504	
/round/ end	200/503	RW-RGS-011 / RW-RGS-008 /	SERVICE_BUSY	sì	
/wallet/ debit	credit (Casino)	200/503	RW- CAS-001,002,006,	LIMITS/ INSUFFICIENT_FUND	sì su
/limits/ check	200/503	RW-CAS-003 / RW-RGS-008 /	LIMITS/ SERVICE_BUSY	sì su 503	
Webhook playerExcluded	balanceChanged (Casino→RGS)	202/503	RW-CAS-007 / RW-NET-009 / RW-SYS-000	SERVICE_BUSY	sì su 50 3 (re